



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS

A MINOR PROJECT PROPOSAL ON AI-POWERED SMART
GYM ASSISTANT SYSTEM

BY
ANIKET NAYAK(PUR080BCT013)
BIBEK SODARI(PUR080BCT025)
GURU PRAKASH GUPTA(PUR080BCT032)

DEPARTMENT OF ELECTRONICS AND COMPUTER
ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL

AUGUST, 2026



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS, DHARAN

DEPARTMENT OF ELECTRONICS AND COMPUTER
ENGINEERING

LETTER OF APPROVAL

This undersigned certify that we have read, and recommended to the Department of Electronics and Computer Engineering, Purwanchal Campus, a minor project report entitled “AI-POWERED SMART GYM ASSISTANT SYSTEM” submitted by Aniket Nayak, Bibek Sodari, and Guru Prakash Gupta. The report has been examined and is submitted in partial fulfillment of the academic requirements for the completion of the Bachelor’s Degree in Computer Engineering.

Er. Manoj Kumar Guragain
Head of Department
Department of Electronics and Computer
Engineering,
Purwanchal Campus, Dharan

Date of Approval: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Purwanchal Campus, for giving us the opportunity to work on this project. We are especially thankful to our faculty teacher.

We are also grateful to our Head of Department, Asst. Prof. Manoj Kumar Guragain, and to all the teachers of the department who supported us with their knowledge and technical resources.

Finally, we would like to thank our friends and family for their constant support and motivation, which helped us complete this project successfully.

Any suggestions or criticisms will be highly appreciated and acknowledged.

Aniket Nayak (PUR080BCT013)

Bibek Sodari (PUR080BCT025)

Guru Prakash Gupta (PUR080BCT032)

ABSTRACT

The Smart Gym Assistant is a low-cost wearable exercise-monitoring system designed to recognize exercise movements, identify posture quality, count repetitions, and provide immediate feedback. The system uses two MPU6050 inertial sensors connected to an ESP32 microcontroller. Acceleration and gyroscope readings from both sensors form a twelve-feature input vector for a compact K-nearest neighbors classifier.

The ESP32 standardizes the sensor features, performs local KNN inference, and uses batch voting to produce stable exercise and posture predictions. An OLED display and buzzer provide local feedback, while a Wi-Fi connection sends collection data and live results to a FastAPI service and browser dashboard. The training pipeline cleans labeled CSV data, evaluates prototype counts and neighbor values, and exports the selected model to an embedded header file.

The evaluated classes include bicep bad, bicep good, bicep idle, squat bad, squat good, and squat idle. The results show strong recognition of idle classes, with F1-scores above 94%, while posture classes remain more challenging because good and bad movement patterns can overlap. The system demonstrates that wearable inertial sensing and lightweight machine learning can provide practical exercise feedback on resource-constrained hardware.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	ix
1 INTRODUCTION	1
1.1 Background	1
1.2 Problem Definition	1
1.3 Objectives	1
1.4 Scope of Project	1
2 RELATED THEORY	3
2.1 Artificial Intelligence and Machine Learning	3
2.2 Human Activity Recognition	3
2.3 Inertial Motion Sensing	3
2.4 Feature Scaling and Standardization	4
2.5 K-Nearest Neighbors Classification	4
2.6 Live Batch Prediction	5
2.7 Edge Computing with ESP32	5
2.8 ESP32 and Real-Time Task Execution	5
2.9 I2C Communication and Multiple Sensors	6
2.10 Double Buffering and Data Integrity	6

2.11	Collection Mode and Live Mode	7
2.12	HTTP API and WebSocket Monitoring	7
2.13	Real-Time Feedback and System Interface	8
2.14	Repetition Counting Algorithm	8
2.15	Model Evaluation	9
3	LITERATURE REVIEW	11
3.1	Wearable Sensor-Based Activity Recognition	11
3.2	Inertial Sensors for Exercise Monitoring	11
3.3	Evolution of AI in Fitness Monitoring	12
3.4	Machine Learning Methods for Activity Classification	12
3.5	Deep Learning-Based Recognition	13
3.6	Posture Detection and Real-Time Feedback	13
3.7	Data Processing and Feature Engineering	13
3.8	Repetition Counting and Movement Segmentation	14
3.9	Embedded and Connected Fitness Systems	14
3.10	Research Gap and Proposed Contribution	15
4	METHODOLOGY	16
4.1	Overview	16
4.2	Feasibility Analysis	17
4.2.1	Technical Feasibility	17
4.2.2	Economic Feasibility	17
4.2.3	Operational Feasibility	17
4.3	Requirement Analysis	17

4.3.1	Functional Requirements	17
4.3.2	Non-Functional Requirements	18
4.4	System Block Diagram	18
4.4.1	Data Collection	19
4.4.2	Detection Engine	20
4.4.3	Response System	20
4.5	Data Flow for Data-Collection Mode	20
4.6	Data Flow for Live Mode	21
4.7	Block Diagram for Training the KNN Model	22
4.7.1	Training Dataset	24
4.7.2	Model Evaluation	24
4.8	Application Deployment	25
4.9	System Evaluation	25
5	EXPERIMENTAL SETUP	27
5.1	Sensor Configuration	27
5.2	Hardware Configuration	27
5.3	Software Environment	27
5.4	Data Collection Setup	27
6	SYSTEM DESIGN	29
6.1	Data Flow Diagram	29
6.2	Use-Case Diagram	30
6.3	Sequence Diagram	31
6.4	Activity Diagram	31

6.5	Class Diagram	32
7	RESULTS AND DISCUSSION	33
7.1	KNN Parameter Evaluation	33
7.2	Class-Wise Performance	34
7.3	Discussion	35
8	CONCLUSION	37
9	LIMITATIONS AND FUTURE ENHANCEMENTS	38
9.1	Limitations	38
9.2	Future Improvements	38
9.2.1	Support for Additional Exercises	38
9.2.2	Training with More Users and Motion Styles	39
9.2.3	Improved Posture Detection	39
9.2.4	Optimized Embedded Model	39
9.2.5	Offline Data Buffering and Reliable Communication	39
9.2.6	User Interface Enhancements	39
	REFERENCES	40

LIST OF FIGURES

Figure 4.1: System design diagram of the Smart Gym Assistant.	16
Figure 4.2: Conceptual block diagram of the Smart Gym Assistant. . .	19
Figure 4.3: Data-collection mode flow.	21
Figure 4.4: Live-mode flow.	22
Figure 4.5: KNN model training pipeline.	23
Figure 4.6: Application deployment diagram.	25
Figure 6.1: Level 0 data flow diagram of the Smart Gym Assistant. . .	29
Figure 6.2: Level 1 data flow diagram of the Smart Gym Assistant. . .	30
Figure 6.3: Use-case diagram of the Smart Gym Assistant.	30
Figure 6.4: Sequence diagram for collection and live monitoring.	31
Figure 6.5: Activity diagram of the Smart Gym Assistant workflow. . .	32
Figure 6.6: Class diagram of the Smart Gym Assistant software structure.	32
Figure 7.1: KNN validation accuracy for different parameter values. . .	33
Figure 7.2: KNN parameter-performance heatmap.	34
Figure 7.3: Normalized validation confusion matrix.	35

LIST OF TABLES

Table 4.1: Functional requirements	18
Table 4.2: Non-functional requirements	18

LIST OF ABBREVIATIONS

AI	: Artificial Intelligence
API	: Application Programming Interface
CSV	: Comma-Separated Values
DFD	: Data Flow Diagram
ESP32	: Espressif Systems 32-bit Microcontroller
FastAPI	: Fast Application Programming Interface
FR	: Functional Requirement
HTTP	: Hypertext Transfer Protocol
I2C	: Inter-Integrated Circuit
IDE	: Integrated Development Environment
IMU	: Inertial Measurement Unit
JSON	: JavaScript Object Notation
KNN	: K-Nearest Neighbors
ML	: Machine Learning
MPU6050	: Motion Processing Unit 6050
NFR	: Non-Functional Requirement
OLED	: Organic Light-Emitting Diode
Wi-Fi	: Wireless Fidelity

CHAPTER 1

INTRODUCTION

1.1 Background

Regular exercise contributes to a healthy lifestyle, but beginners often struggle to maintain the correct form and keep an accurate record of their progress. Repeated movements performed with poor posture can make a workout less effective and may also increase the risk of injury. This project responds to that practical problem by developing a smart gym assistant that uses motion sensors and machine learning to recognize exercises, count repetitions, and identify posture errors during exercise.

1.2 Problem Definition

Many gym users perform exercises without a trainer continuously checking their form, and repetitions are often recorded manually. As a result, posture mistakes may go unnoticed and workout records may be incomplete or inaccurate. The project therefore aims to provide an accessible AI-based assistant that can track exercises and give posture-correction feedback using affordable embedded hardware.

1.3 Objectives

- To develop a smart gym assistant that monitors and analyzes exercise movements in real time.
- To recognize supported exercises, count repetitions, identify incorrect posture, and provide feedback that helps users improve their form and reduce the risk of injury.

1.4 Scope of Project

- The current implementation focuses on squats and bicep curls using two MPU6050 motion sensors and a KNN model.

- The system identifies the exercise, detects good or bad posture, counts repetitions, and provides feedback through an OLED, buzzer, and web interface.
- The same data and model pipeline can later be extended to exercises such as push-ups by collecting and labeling additional training data.

CHAPTER 2

RELATED THEORY

2.1 Artificial Intelligence and Machine Learning

Artificial intelligence (AI) refers to systems that can perceive information, learn from it, and use that learning to make decisions. Machine learning (ML) is a part of AI in which the system learns a relationship from examples instead of being controlled entirely by hand-written rules [1–4].

In this project, ML is used to distinguish bicep curls and squats and to assess their posture from motion-sensor measurements. Each recorded sample is paired with a label such as `bicep_good`, `bicep_bad`, `squat_good`, or `squat_bad`. During live operation, a new sensor sample is compared with labeled training examples and the closest movement pattern is selected. The same label structure can later be extended to exercises such as push-ups without changing the basic KNN pipeline.

2.2 Human Activity Recognition

Human activity recognition (HAR) means identifying a person’s activity from measured signals. Wearable inertial sensors are useful for HAR because they capture acceleration and angular motion without requiring a camera or a fixed environment [5–9]. For an exercise assistant, HAR can be extended from identifying an exercise to identifying the quality of its execution. In the current implementation, the exercise set contains bicep curls and squats; the exercise name and posture condition are represented together in the class label so additional exercises can be added later.

2.3 Inertial Motion Sensing

An inertial measurement unit (IMU) provides measurements related to linear acceleration and angular velocity. The project uses two MPU6050 devices so that motion can be observed from two sensor locations. For every sampling instant,

the ESP32 records three acceleration values and three gyroscope values from each sensor. The resulting feature vector is

$$\mathbf{x} = [a_{x1}, a_{y1}, a_{z1}, g_{x1}, g_{y1}, g_{z1}, a_{x2}, a_{y2}, a_{z2}, g_{x2}, g_{y2}, g_{z2}]. \quad (2.1)$$

The two-sensor arrangement provides more information than a single acceleration axis or a single sensor. It can help distinguish similar exercise movements when the relative orientation or motion of the two body locations is important. Sensor data must nevertheless be collected with consistent placement, orientation, sampling interval, and exercise labeling to obtain reliable predictions.

2.4 Feature Scaling and Standardization

Acceleration and gyroscope measurements have different numerical ranges. If raw values are compared directly, features with larger magnitudes can dominate the distance calculation. The training process therefore standardizes each feature using its training-set mean and standard deviation:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \quad (2.2)$$

where μ_i is the mean and σ_i is the standard deviation of feature i . The same values calculated during training are embedded in `knn_model.h` and used by the ESP32 during inference. This keeps training and live prediction in the same feature space.

2.5 K-Nearest Neighbors Classification

K-nearest neighbors (KNN) is a supervised, instance-based classification method. For an unseen feature vector, KNN calculates its distance from stored labeled examples, selects the k closest examples, and assigns the majority class among those neighbors [2].

Using standardized features, the squared Euclidean distance between a live vector and a stored prototype is

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{12} (x_i - y_i)^2. \quad (2.3)$$

The embedded model uses a fixed value of $k = 5$. To fit the model within the ESP32’s memory and computation limits, representative prototypes are selected for each label. The prototypes, labels, feature means, and feature scales are stored in program memory. This avoids requiring a full Python or scikit-learn runtime on the microcontroller. The use of a microcontroller for sensing and control is consistent with the embedded prototyping principles described by Banzi and Shiloh [10].

2.6 Live Batch Prediction

An individual sensor reading may be noisy or may not contain enough information to identify the movement confidently. To make the result more stable, the firmware collects 50 samples at approximately 50 samples per second. KNN is applied to each sample, and the label receiving the most votes becomes the batch prediction. The displayed confidence is calculated as

$$\text{confidence} = \frac{\text{votes for winning label}}{\text{number of samples in batch}}. \quad (2.4)$$

This temporal voting reduces the effect of a single unstable reading and provides a practical live update approximately once per second.

2.7 Edge Computing with ESP32

Edge computing performs data processing close to the sensing device instead of sending every raw measurement to a remote service. In this project, the ESP32 reads the sensors, performs feature preparation and KNN inference, and generates immediate feedback locally. The sensor buffers are uploaded to the FastAPI server for storage and web monitoring. This division reduces the dependency of the classification step on network latency while retaining a central record of collected data.

2.8 ESP32 and Real-Time Task Execution

The ESP32 is a microcontroller platform suitable for connected sensing applications because it combines processing, wireless networking, general-purpose input/output, and real-time task support in a compact device [10, 11]. The firmware separates continuous sensor collection from network processing. A collector task reads both MPU6050 devices at a fixed sampling interval, while an uploader task processes completed buffers. This separation prevents a slow HTTP request from directly blocking the sensor sampling loop.

The task arrangement follows a producer-consumer model. The collector is the producer of sensor samples and the uploader is the consumer of completed batches. Shared buffer indexes and queue operations are protected by a mutex and a queue so that the two tasks can operate safely on different ESP32 cores. The design is particularly useful for live monitoring, where sampling regularity is more important than waiting for each network operation to finish.

2.9 I2C Communication and Multiple Sensors

Inter-Integrated Circuit (I2C) is a synchronous two-wire communication protocol that uses a shared serial data line (SDA) and serial clock line (SCL). Devices on the same bus are distinguished by their addresses. The MPU6050 provides acceleration and gyroscope measurements through I2C, and its address can be selected using the AD0 pin [12].

This project assigns address 0x68 to the first MPU6050 and 0x69 to the second one. Both sensors can therefore be read through the same ESP32 I2C bus without requiring separate data buses. Correct common grounding, power supply, pull-up behavior, and AD0 configuration are necessary for reliable communication. The OLED display also uses I2C, but has a different address, allowing it to share the bus with the motion sensors.

2.10 Double Buffering and Data Integrity

The firmware uses two fixed-size sensor buffers in a ping-pong arrangement. While the collector fills one buffer, the uploader can process the other buffer. When a buffer reaches 50 samples, it is placed in an upload queue and the collector changes to the alternate buffer. This technique is called double buffering and is commonly used when a producer must continue generating data while a consumer performs a slower operation.

Each uploaded sample contains a timestamp and the twelve sensor features. A generation number identifies the collection session, so a buffer belonging to an old session can be discarded after reset. If the upload queue becomes full, the firmware drops an older pending batch instead of stopping the collector. This keeps the live sampling path responsive, although it means that network congestion can cause some batches not to be stored.

2.11 Collection Mode and Live Mode

The system has two operation modes. In data-collection mode, labeled sensor batches are sent to the FastAPI server and appended to the training CSV file. The selected exercise and posture fields provide the ground-truth label for those recordings. In live mode, the ESP32 performs KNN inference locally and sends only the latest result to the live-result endpoint; raw live samples are not appended to the training file.

Separating these modes prevents live predictions from accidentally becoming training labels. It also allows the same hardware to be used first for supervised data collection and later for independent evaluation. The live mode reports the predicted exercise, posture, confidence, timestamp, and sample count.

2.12 HTTP API and WebSocket Monitoring

The ESP32 communicates with the FastAPI service using HTTP. Data collection uses a JSON POST request containing session information and an array of sensor

samples. Live mode uses an HTTP request for the prediction result. FastAPI validates the request fields, stores collection batches in CSV format, and maintains the latest live status in memory.

For browser monitoring, the service exposes both a live-status endpoint and a WebSocket channel. HTTP polling is simple and suitable for periodic status checks, whereas a WebSocket can push a new prediction to connected clients as soon as the ESP32 publishes it. The dashboard can therefore show the exercise, posture, confidence, repetition count, mode, and online state without reading the ESP32's serial output.

2.13 Real-Time Feedback and System Interface

The predicted exercise, posture, and confidence are shown on the OLED display. The display can report good, bad, or idle; it also reports paused and upload-failure states. A buzzer can provide an additional local signal, such as a short tone for acceptable posture and a longer tone for an incorrect posture. The FastAPI service receives the prediction together with the sensor batch, and the browser dashboard polls the latest status to show exercise name, posture, confidence, and connection state.

2.14 Repetition Counting Algorithm

Exercise classification and repetition counting are related but different tasks. The classifier identifies the exercise and posture class, while the repetition counter identifies completed movement cycles. Repetition counting from wearable sensors can be treated as a periodic-signal analysis problem because repeated movements produce recurrent patterns in acceleration and angular velocity [13–15].

First, the acceleration and gyroscope components can be combined into motion magnitudes to reduce sensitivity to sensor-axis orientation. For sensor j , the acceleration magnitude is

$$a_j(t) = \sqrt{a_{xj}(t)^2 + a_{yj}(t)^2 + a_{zj}(t)^2}, \quad (2.5)$$

and the gyroscope magnitude is

$$g_j(t) = \sqrt{g_{xj}(t)^2 + g_{yj}(t)^2 + g_{zj}(t)^2}. \quad (2.6)$$

The magnitudes from both sensors can be averaged or combined into a motion signal $m(t)$. A short moving-average filter reduces high-frequency sensor noise:

$$\tilde{m}(t) = \frac{1}{w} \sum_{i=0}^{w-1} m(t-i), \quad (2.7)$$

where w is a small window selected according to the sampling rate.

The counter uses an adaptive baseline and two thresholds. A movement is considered active when the filtered signal moves above the activation threshold, and a repetition is counted only after the signal returns below the release threshold. Using separate thresholds, known as hysteresis, prevents small fluctuations near one threshold from producing multiple counts. A minimum time interval between counts also prevents a single movement from being counted more than once.

The live repetition-counting procedure is:

1. Read the two IMUs and calculate the combined motion signal.
2. Apply a moving-average filter and update the recent baseline.
3. Detect a rising crossing of the activation threshold.
4. Enter the MOVING state and wait for a falling crossing of the release threshold.
5. If the minimum repetition interval has elapsed, increment the repetition count.
6. Return to the READY state and wait for the next movement cycle.

This state-machine design is suitable for an ESP32 because it requires only a small rolling buffer and a few scalar variables. The exercise class predicted by KNN can select exercise-specific thresholds, since a squat and a bicep curl may have different signal amplitudes and cycle durations. The counter should be reset when a new exercise is selected, when the user pauses, or when the system detects a long idle

interval. A repetition is counted only after a complete active-and-release cycle, which avoids counting partial movements as valid repetitions.

For a more reliable implementation, the threshold and minimum interval should be estimated from labeled training recordings rather than fixed only from one user’s motion. Validation should compare the automatic count with manually counted repetitions using count error and mean absolute error in addition to classification accuracy.

2.15 Model Evaluation

Accuracy alone can be misleading when classes have different numbers of samples. The model is therefore evaluated using a stratified validation split, in which each label is represented in both the training and validation sets. A confusion matrix shows the relationship between actual and predicted labels. Precision, recall, and F1-score are also useful for posture detection because they reveal whether a class is being missed or incorrectly predicted:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad (2.8)$$

$$F1 = 2 \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \quad (2.9)$$

The training notebook compares different values of k and different numbers of embedded prototypes per label using validation accuracy, a parameter-performance graph, and a normalized confusion matrix.

CHAPTER 3

LITERATURE REVIEW

3.1 Wearable Sensor-Based Activity Recognition

Researchers have increasingly explored wearable sensors as an alternative to camera-based activity monitoring. Accelerometers and gyroscopes are attractive for this purpose because they are inexpensive, portable, and able to record movement without restricting the user to a particular camera view. Surveys by Wang et al. and Lara and Labrador discuss how these sensors are used for activity recognition, while also pointing out the effects of sensor placement, signal quality, and differences between users [5, 7].

Kwapisz et al. demonstrated that acceleration data can be used to identify activities using a mobile device, showing the usefulness of time-domain inertial signals for activity classification [6]. Similarly, Bao and Intille investigated user-annotated acceleration data and showed that sensor location and the selected activity features influence recognition performance [13]. These findings support the use of two MPU6050 sensors in the present project, since measurements from two locations can provide more information about exercise movement and posture than a single sensor.

3.2 Inertial Sensors for Exercise Monitoring

Bulling et al. presented a tutorial on human activity recognition using body-worn inertial sensors and discussed the importance of sensor orientation, segmentation, and feature representation [8]. These issues are directly relevant to gym-exercise monitoring. A sensor attached at a consistent body location can record acceleration and angular velocity patterns associated with the different phases of a squat or bicep curl.

In this project, two MPU6050 devices are used to capture three-axis acceleration and three-axis gyroscope measurements. Each reading therefore produces a 12-value feature vector. This arrangement avoids the privacy and viewpoint lim-

itations of video-based systems while keeping the hardware simple enough for a low-cost gym assistant prototype.

3.3 Evolution of AI in Fitness Monitoring

Early fitness monitoring systems mainly depended on manual workout records, pedometers, and simple threshold rules. The development of wearable sensors and embedded computing enabled continuous collection of physiological and motion data. Modern AI systems use these measurements to recognize exercises, analyze posture, count repetitions, and provide feedback during a workout. The AI principles of perception, learning, and decision making described by Russell and Norvig provide a general foundation for these intelligent fitness applications [1].

Recent machine-learning and deep-learning methods can learn increasingly complex movement patterns from labeled sensor datasets. However, a fitness assistant for a resource-constrained ESP32 must also consider memory, processing time, power, and network availability. This project therefore uses a compact KNN model for local inference while retaining a design that can be extended to more advanced models in future work.

3.4 Machine Learning Methods for Activity Classification

Activity-recognition systems have used decision trees, support-vector machines, nearest-neighbor methods, neural networks, and deep-learning models. Geron describes the importance of preparing features, training a classifier, and evaluating it on data that was not used during training [2]. McKinney highlights the importance of reliable data loading, cleaning, analysis, and visualization when building a data-driven system [3].

K-nearest neighbors was selected for this project because it is straightforward to train, works directly with labeled feature vectors, and does not require a large runtime on the ESP32. The training process standardizes the sensor features and stores representative examples from each class in program memory. During live use, the closest examples determine the exercise and posture prediction. This gives

the prototype a model that is easy to inspect and practical to run on the device.

3.5 Deep Learning-Based Recognition

Deep learning has achieved strong results in many perception and activity-recognition tasks. LeCun, Bengio, and Hinton describe how multilayer neural networks can learn representations directly from data [4]. Ordonez and Roggen used convolutional and recurrent neural networks for multimodal wearable activity recognition, demonstrating the potential of temporal neural models for sensor sequences [9].

Although deep learning is a possible future direction, it is not the primary method used in the current prototype. Neural-network training generally requires more data, computation, and memory than the selected compact KNN implementation. KNN also makes it easier to inspect the relationship between a live sensor vector and the stored training examples. A future version could compare the embedded KNN model with a quantized neural network after collecting a larger and more diverse dataset.

3.6 Posture Detection and Real-Time Feedback

Posture correction is an important part of an intelligent fitness assistant because incorrect form can reduce exercise effectiveness and increase injury risk. A posture classifier compares live acceleration and angular-velocity patterns with labeled good- and bad-posture examples. The output can then be converted into immediate visual or audio feedback.

In the proposed system, the KNN label contains both the exercise and posture state. The ESP32 displays the predicted exercise, good, bad, or idle state on an OLED and can activate a buzzer for incorrect posture. The web dashboard provides the same prediction and confidence for remote monitoring. This combines recognition and feedback in the same low-cost embedded workflow.

3.7 Data Processing and Feature Engineering

Raw accelerometer and gyroscope readings may contain noise, drift, and fluctuations caused by sensor placement or user movement. Data preparation is therefore required before model training. Standardization, consistent sampling, removal of malformed rows, and validation of class labels improve the reliability of the training data [3].

Common motion features include mean acceleration, standard deviation, signal magnitude, angular velocity, and orientation-related values. The current prototype uses the twelve direct readings from the two MPU6050 sensors and standardizes them before calculating KNN distances. This keeps the embedded implementation compact, while the feature pipeline can later be expanded if additional exercises or users require greater separation between classes.

3.8 Repetition Counting and Movement Segmentation

Activity recognition identifies what a user is doing, whereas repetition counting determines how many complete movement cycles have been performed. Ravi et al. and Bao and Intille show that repeated activities can be recognized from recurring patterns in accelerometer signals [13, 14]. Signal processing methods such as smoothing, thresholding, peak detection, and hysteresis can therefore be used to segment exercise cycles before incrementing a repetition counter [15].

In this project, repetition counting is designed as a separate processing stage from KNN classification. The classifier identifies the exercise and posture, while the counter uses a filtered motion magnitude and a state transition from an active phase to a release phase. This separation allows the counting thresholds to be adjusted for each exercise and prevents a noisy single sample from being interpreted as a complete repetition.

3.9 Embedded and Connected Fitness Systems

Arduino-based embedded prototyping provides a practical platform for connecting sensors, displays, actuators, and network services [10]. The ESP32 extends this approach with wireless communication and real-time task support [11]. FastAPI provides a lightweight service layer for receiving structured sensor batches and serving live status to a browser [16].

The proposed system combines these ideas into one workflow. The ESP32 collects data from two IMUs, performs local KNN inference in live mode, displays the prediction on an OLED, and uses a buzzer for posture feedback. In collection mode, labeled samples are transmitted to the FastAPI service for storage and later model training. The browser dashboard presents the latest exercise, posture, confidence, and repetition information to the user.

3.10 Research Gap and Proposed Contribution

Existing activity-recognition research demonstrates the value of wearable inertial sensors and machine learning, but many solutions focus on general activities, offline analysis, or computationally larger models. Other systems may identify an exercise without explicitly providing posture feedback, repetition counting, and a live user interface in one low-cost prototype.

This project addresses that gap by integrating two MPU6050 sensors, compact KNN classification, batch voting for stable live predictions, repetition-counting logic, OLED and buzzer feedback, and a FastAPI web monitor. The current evaluation focuses on two exercises, bicep curls and squats. The design remains extensible: additional exercises such as push-ups can be added by collecting labeled samples, including their classes during training, and regenerating the embedded model.

CHAPTER 4

METHODOLOGY

4.1 Overview

The Smart Gym Assistant is developed as an embedded sensing and machine-learning system. Two MPU6050 sensors collect body-motion data, and an ESP32 processes the readings to identify supported exercises and posture states. The system also counts completed repetitions and provides feedback through an OLED display, a buzzer, and a web dashboard. The current implementation focuses on bicep curls and squats.

The methodology covers the feasibility of the system, its requirements, data flow, KNN model-training pipeline, live operation, application deployment, and evaluation. The design is modular so additional exercises can be added later by collecting new labeled data and regenerating the embedded model.

Figure 4.1 presents the overall system design of the Smart Gym Assistant, including the sensing, processing, communication, storage, and output components.

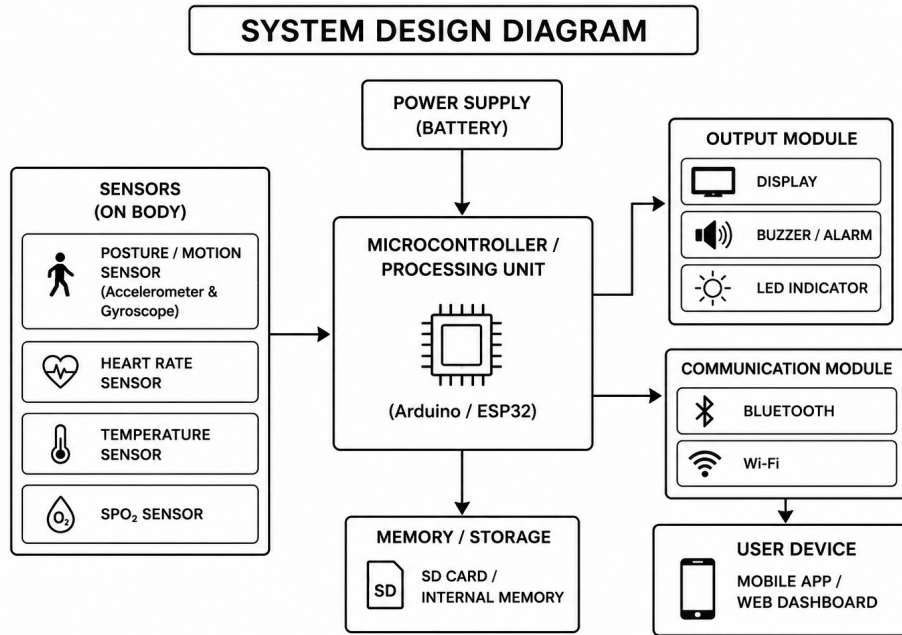


Figure 4.1: System design diagram of the Smart Gym Assistant.

4.2 Feasibility Analysis

4.2.1 Technical Feasibility

The system is technically feasible because the ESP32 can communicate with two MPU6050 sensors over I2C, process a compact KNN model, connect to Wi-Fi, and drive the OLED and buzzer. The model uses twelve numeric sensor features and a limited set of representative prototypes, which keeps live inference practical on the microcontroller.

4.2.2 Economic Feasibility

The prototype uses low-cost and widely available components: an ESP32 board, two MPU6050 modules, an OLED display, and a buzzer. The software tools are open-source or freely available for educational use. Therefore, the system can be developed and maintained at a lower cost than camera-based commercial fitness systems.

4.2.3 Operational Feasibility

The system is simple to operate. The user places the sensors consistently, connects the ESP32 to Wi-Fi, and selects either data-collection mode or live mode. Results are shown locally on the OLED and remotely through the browser dashboard. The feedback does not require a dedicated camera or continuous supervision by a trainer.

4.3 Requirement Analysis

4.3.1 Functional Requirements

Table 4.1 lists the main functions required from the system.

Table 4.1: Functional requirements

ID	Requirement
FR1	Collect acceleration and gyroscope readings from two MPU6050 sensors.
FR2	Identify the supported exercise and posture using the trained KNN model.
FR3	Count completed repetitions from the motion signal.
FR4	Display exercise, posture, confidence, and system status on the OLED.
FR5	Provide buzzer feedback when bad posture is detected.
FR6	Collect and upload labeled sensor batches in data-collection mode.
FR7	Publish live predictions to the FastAPI service and web dashboard.
FR8	Store labeled training samples in CSV format.

4.3.2 Non-Functional Requirements

Table 4.2 lists the quality and operational requirements of the system.

Table 4.2: Non-functional requirements

ID	Requirement
NFR1	Sample motion data at approximately 50 samples per second.
NFR2	Provide a live result after each 50-sample batch.
NFR3	Continue sampling while a network request is being processed.
NFR4	Use portable, low-cost, and easily replaceable hardware.
NFR5	Provide clear local and web-based feedback.
NFR6	Allow future exercise classes to be added without redesigning the complete system.

4.4 System Block Diagram

Figure 4.2 presents the conceptual processing flow. Motion data is collected by the sensors and passed to the ESP32 for preprocessing. The KNN model identifies the exercise and posture, while the repetition counter and feedback system provide the final user response.

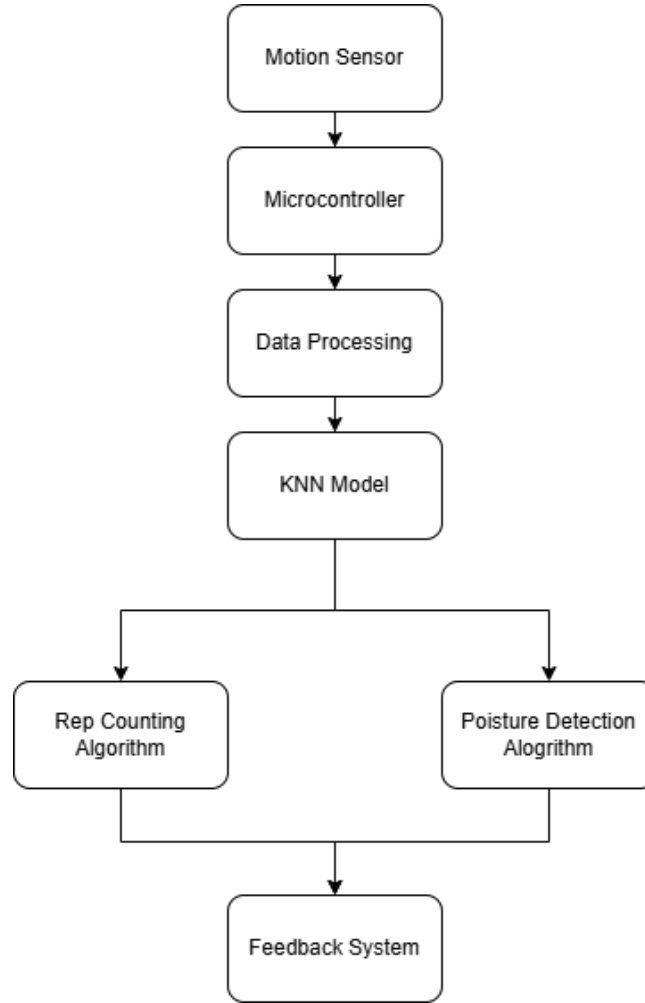


Figure 4.2: Conceptual block diagram of the Smart Gym Assistant.

The original conceptual figure uses the general terms “motion sensor” and “microcontroller.” In the implemented prototype, these correspond to two MPU6050 modules and the ESP32. The implemented output devices are the SH1106 OLED and the buzzer; the web dashboard is reached through the ESP32 Wi-Fi connection and FastAPI.

4.4.1 Data Collection

The two MPU6050 sensors are connected to the ESP32 through the I2C bus. The first sensor uses address 0x68 and the second uses address 0x69. The AD0 pins are configured differently so both sensors can share the same SDA and SCL lines. Each reading contains three acceleration and three gyroscope values from each

sensor, forming the twelve-feature vector

$$\mathbf{x}(t) = [a_{x1}, a_{y1}, a_{z1}, g_{x1}, g_{y1}, g_{z1}, a_{x2}, a_{y2}, a_{z2}, g_{x2}, g_{y2}, g_{z2}]. \quad (4.1)$$

The collector task samples the sensors every 20 milliseconds. Each sample is timestamped and written to the active buffer. After 50 samples, the completed buffer is passed to the processing or upload task.

4.4.2 Detection Engine

The detection engine standardizes the twelve features and compares the live vector with the stored KNN prototypes. The squared Euclidean distance is

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^{12} (x_i - y_i)^2. \quad (4.2)$$

The five nearest prototypes vote for the sample label. The majority label across a 50-sample batch becomes the final live prediction. The label combines exercise and posture, such as bicep_good, bicep_bad, squat_good, or squat_bad.

4.4.3 Response System

The response system displays the detected exercise, posture, and confidence on the OLED. It can activate the buzzer for bad posture. In live mode, the result is also sent to FastAPI, which makes the latest exercise, posture, confidence, and online state available to the web dashboard.

4.5 Data Flow for Data-Collection Mode

Data-collection mode is used to create labeled training data. The user selects the exercise and posture through the ESP32 web interface or the serial command. The collector then reads both sensors, creates timestamped samples, and stores them in alternating buffers.

When a buffer reaches 50 samples, the uploader serializes the batch as JSON and

sends it using an HTTP POST request to the FastAPI endpoint. The payload contains the user, exercise, posture, and sensor samples. FastAPI validates the payload and appends each sample to the labeled CSV file. The resulting data is later used by the Python training script.

Figure 4.3 illustrates the data-collection process.

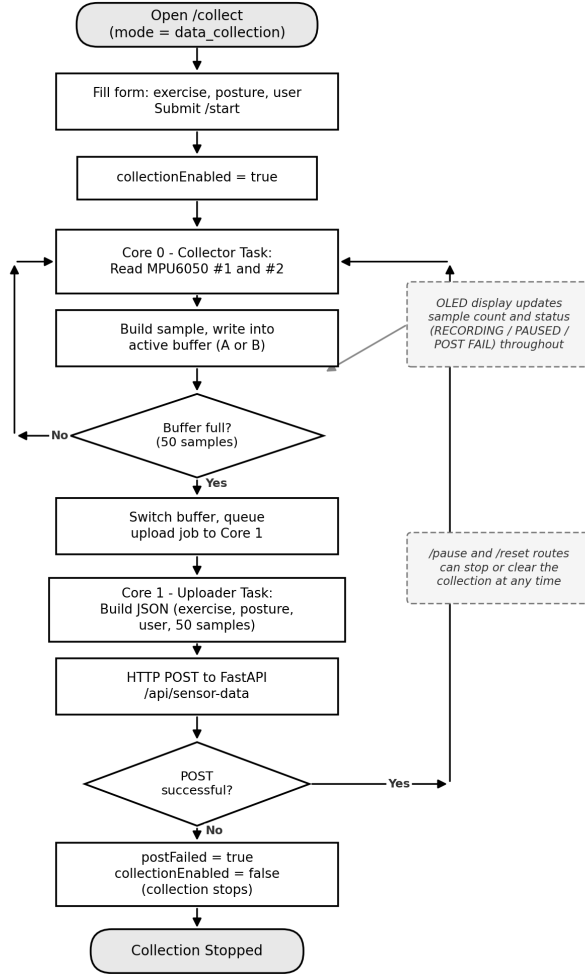


Figure 4.3: Data-collection mode flow.

4.6 Data Flow for Live Mode

Live mode uses the same sensor-acquisition path but does not append raw samples to the training CSV. Once a 50-sample buffer is complete, the ESP32 runs KNN inference for every sample. The label with the highest batch vote is selected, and confidence is calculated as

$$C = \frac{V_{\text{winner}}}{N}, \quad (4.3)$$

where V_{winner} is the winning vote count and N is the number of samples in the batch.

The ESP32 updates the OLED and buzzer locally, prints the result to the serial monitor, and sends the prediction to the live-prediction endpoint. The browser dashboard receives the latest status through the FastAPI service. This mode keeps the classification decision on the ESP32 and prevents predicted labels from being mistaken for ground-truth training labels.

Figure 4.4 illustrates the live prediction process.

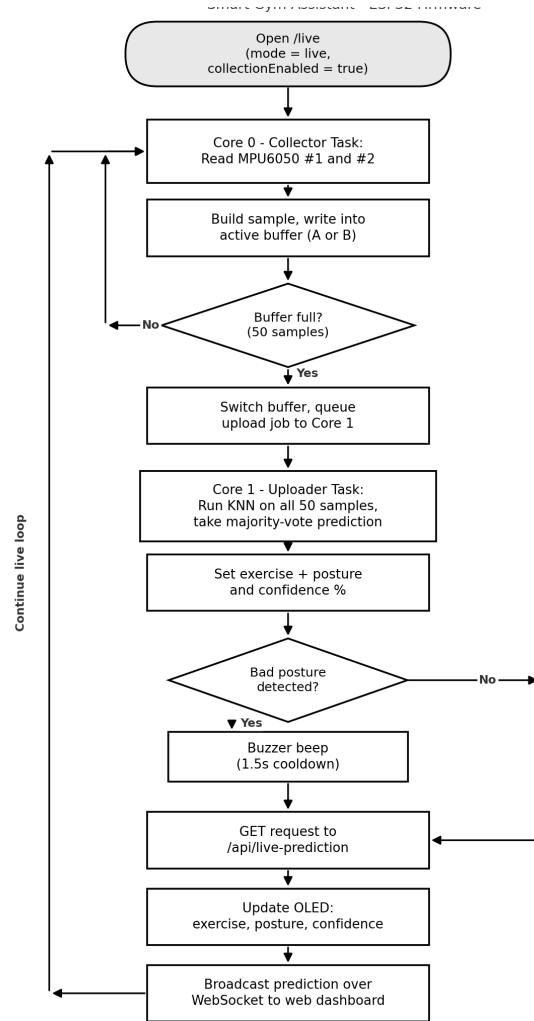


Figure 4.4: Live-mode flow.

4.7 Block Diagram for Training the KNN Model

The KNN training pipeline begins with labeled CSV data and ends with an embedded header file. The major stages are:

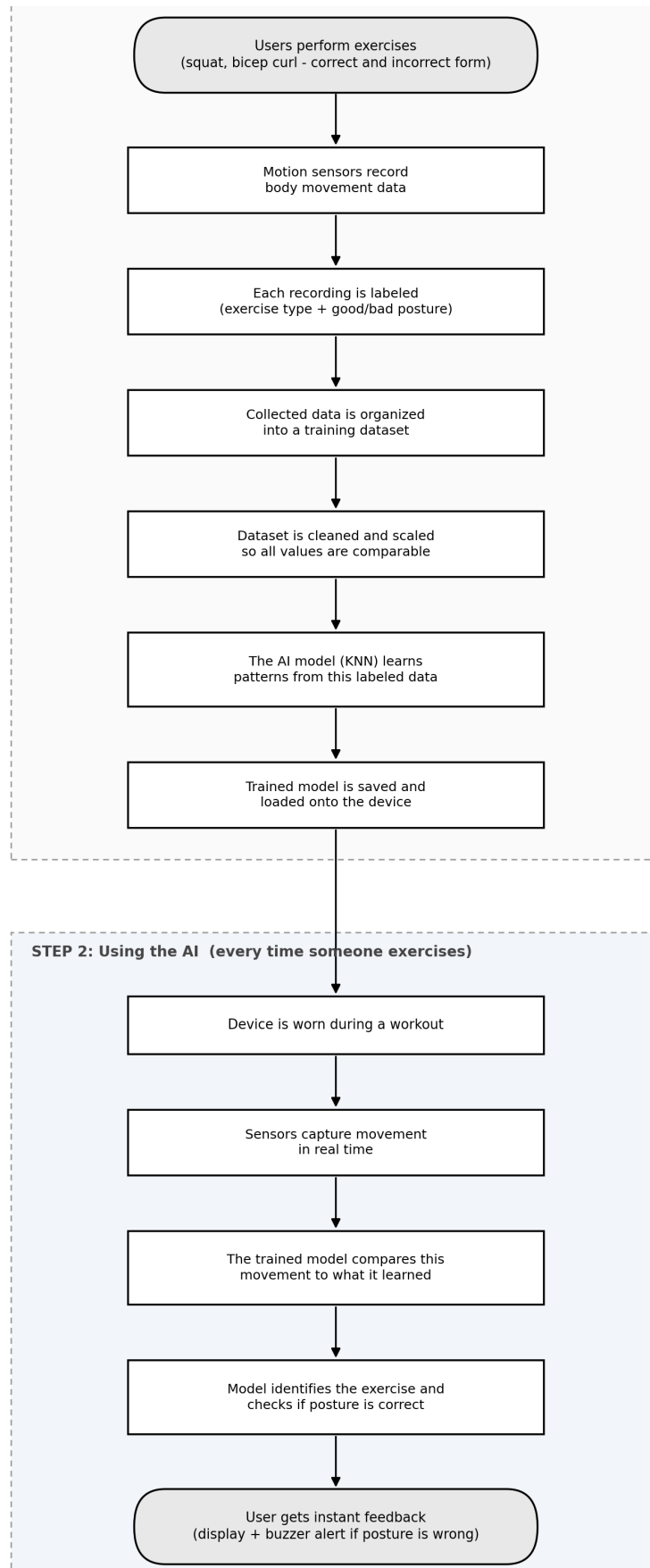


Figure 4.5: KNN model training pipeline.

1. read the selected training CSV;
2. remove repeated headers and invalid sensor rows;
3. separate the twelve sensor features from the metadata and label;
4. calculate the mean and scale of each feature;
5. standardize the feature values;
6. select representative prototypes for every label;
7. evaluate different values of k and prototype counts; and
8. generate knn_model.h for the ESP32.

For a feature x_i , standardization is performed using

$$z_i = \frac{x_i - \mu_i}{\sigma_i}, \quad (4.4)$$

where μ_i and σ_i are calculated from the training prototypes. The generated header stores normalized prototypes, labels, means, and scales in program memory. The ESP32 therefore performs inference without Python or a separate machine-learning library.

4.7.1 Training Dataset

The current dataset is collected directly from the ESP32 and contains timestamped records with user, exercise, posture, combined label, and twelve sensor features. The implemented training scope is bicep curls and squats. Push-ups and other exercises can be supported later by collecting their labeled good and bad posture records and rerunning the training pipeline.

4.7.2 Model Evaluation

The notebook uses a stratified validation split so every available class is represented in training and validation data. It compares multiple values of k and different prototype counts. Performance is reported using accuracy, precision, recall, F1-score, a parameter-performance graph, and a normalized confusion matrix.

4.8 Application Deployment

The ESP32 firmware is compiled and uploaded using Arduino IDE. The sketch contains the sensor collector, KNN model interface, OLED and buzzer logic, Wi-Fi connection, and web-server handlers. The generated `knn_model.h` file must be kept in the same Arduino sketch folder as the firmware files.

The FastAPI application runs on a computer connected to the same local network as the ESP32. The server receives collection batches at `/api/sensor-data`, accepts live predictions at the live-prediction endpoint, and serves the dashboard from the root URL. The ESP32's server address must use the computer's local network IP address rather than `localhost`.

Figure 4.6 illustrates the deployment arrangement.

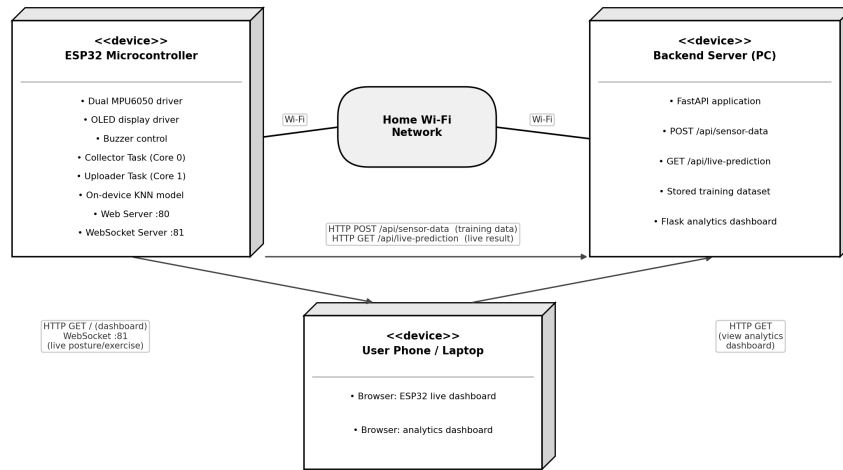


Figure 4.6: Application deployment diagram.

4.9 System Evaluation

The integrated prototype is evaluated in the following order:

1. verify that both MPU6050 sensors respond at their configured I2C addresses;
2. verify that the ESP32 connects to Wi-Fi and reaches the FastAPI server;
3. check that collection batches contain valid timestamps, labels, and twelve features;

4. compare KNN predictions with the known exercise and posture condition;
5. check repetition counts against manually counted movement cycles;
6. verify OLED, buzzer, serial, and dashboard feedback; and
7. measure classification accuracy, posture precision and recall, confidence, update delay, and communication reliability.

This evaluation distinguishes model errors from hardware, network, and user-interface errors. It also provides a clear baseline for improving the model when additional users or exercises are added.

CHAPTER 5

EXPERIMENTAL SETUP

5.1 Sensor Configuration

The experimental setup uses two MPU6050 inertial sensors connected to the ESP32 through the I2C bus. The first sensor uses address 0x68 and the second uses address 0x69. Together, the sensors provide three acceleration and three gyroscope readings each, forming a twelve-feature input vector for the KNN classifier. Samples are collected every 20 milliseconds, giving an approximate sampling rate of 50 samples per second.

5.2 Hardware Configuration

The hardware setup consists of an ESP32 development board, two MPU6050 modules, an SH1106 OLED display, and a buzzer. The ESP32 performs sensor acquisition, feature standardization, KNN inference, repetition counting, and Wi-Fi communication. The OLED displays the exercise, posture, confidence, and system status, while the buzzer provides immediate feedback when bad posture is detected.

5.3 Software Environment

The ESP32 firmware is developed and uploaded using the Arduino IDE. It includes the sensor collector, double-buffered batch processing, embedded KNN model, display and buzzer control, Wi-Fi connection, and web-server handlers. The KNN model is trained with Python from labeled CSV data and exported to `knn_model.h`. A FastAPI application receives collected sensor batches, stores labeled records, accepts live predictions, and provides the web dashboard.

5.4 Data Collection Setup

For each experiment, the sensors are attached consistently to the user's body and the exercise and posture labels are selected before recording. The ESP32 collects 50 samples per batch. In collection mode, each completed batch is serialized as JSON and sent to the FastAPI service, which validates and stores the records in CSV format. The resulting dataset is used to train and evaluate the KNN model for bicep curls and squats.

CHAPTER 6

SYSTEM DESIGN

The Smart Gym Assistant is designed as a connected embedded system that combines motion sensing, edge-based machine-learning inference, local feedback, and browser monitoring. This chapter presents the system from several complementary viewpoints: the data flow, user interactions, time-ordered operation, activity workflow, and software class structure.

6.1 Data Flow Diagram

The Level 0 data flow diagram in Figure 6.1 gives a high-level view of the Smart Gym Assistant as one system. It shows the user and sensor readings entering the system, while classified exercise information, posture feedback, stored data, and dashboard results leave it.

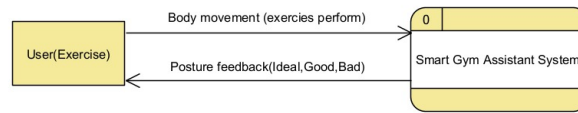


Figure 6.1: Level 0 data flow diagram of the Smart Gym Assistant.

The Level 1 data flow diagram in Figure 6.2 shows how data moves through the complete system. The user provides exercise and posture labels in collection mode, while the two MPU6050 sensors provide motion readings. The ESP32 collects and preprocesses these readings before either uploading labeled batches to the FastAPI service or performing local KNN prediction. The service stores collection data and exposes the latest live result to the browser dashboard.

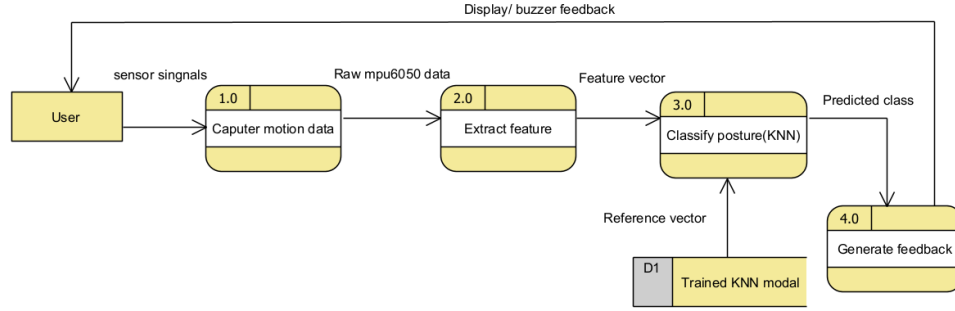


Figure 6.2: Level 1 data flow diagram of the Smart Gym Assistant.

This separation keeps raw training data distinct from live predictions. It also allows the ESP32 to continue making time-sensitive decisions locally while the server handles storage and web monitoring.

6.2 Use-Case Diagram

Figure 6.3 presents the main interactions between the user and the Smart Gym Assistant. The user can configure the exercise and posture, collect labeled sensor data, start live monitoring, and observe exercise, posture, confidence, and repetition results. The system performs sensing, classification, feedback, data storage, and dashboard updates in response to these actions.

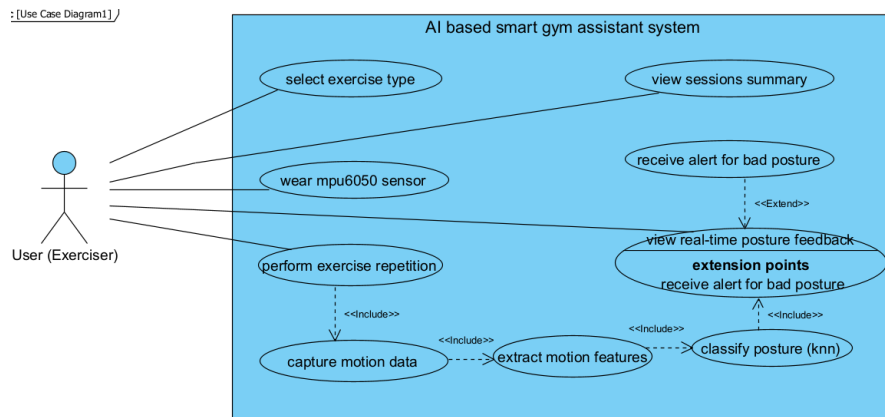


Figure 6.3: Use-case diagram of the Smart Gym Assistant.

The use cases reflect the two operating modes. Collection mode is intended for building the labeled dataset, whereas live mode is intended for real-time exercise and posture feedback.

6.3 Sequence Diagram

The sequence diagram in Figure 6.4 describes the order of communication among the user, sensors, ESP32, FastAPI service, and dashboard. During collection, the ESP32 gathers a complete batch and sends it to the API for validation and storage. During live monitoring, it classifies the batch locally, updates the OLED and buzzer, and publishes the latest result for the dashboard.

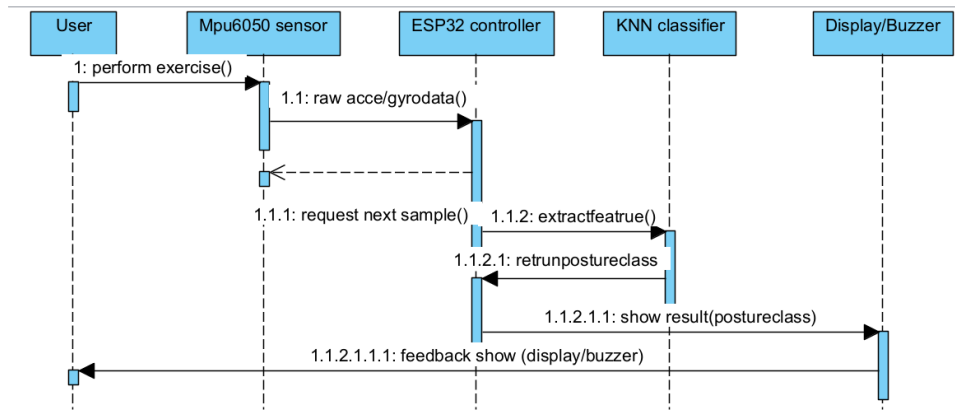


Figure 6.4: Sequence diagram for collection and live monitoring.

The sequence emphasizes that sensor acquisition and inference occur on the ESP32, while the FastAPI service provides storage and monitoring support. This arrangement reduces dependence on network response time for immediate local feedback.

6.4 Activity Diagram

Figure 6.5 represents the operational workflow of the system. The process begins with initialization and sensor setup, followed by mode selection. In collection mode, labeled samples are buffered and uploaded. In live mode, the buffered samples are standardized, classified with KNN, and converted into exercise, posture, confidence, and repetition feedback. The workflow then repeats for the next batch while the system remains active.

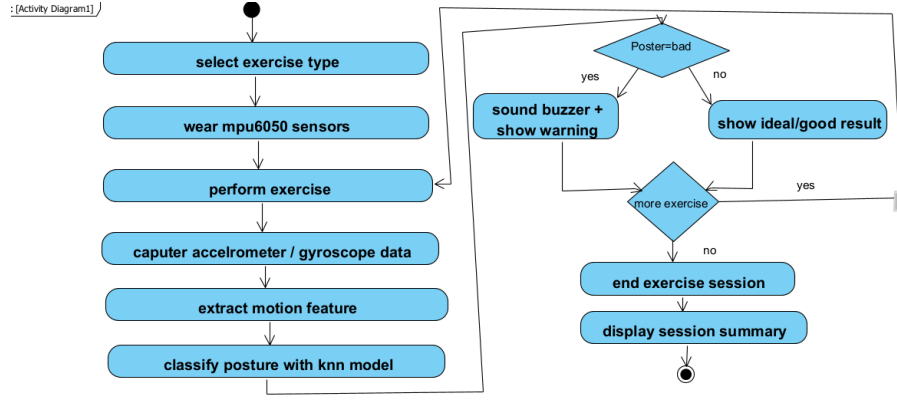


Figure 6.5: Activity diagram of the Smart Gym Assistant workflow.

The activity model also shows the control decisions that prevent collection and live operation from being mixed. This supports reliable labeling and makes the system behavior easier to test.

6.5 Class Diagram

The class diagram in Figure 6.6 summarizes the logical software structure. Sensor and buffer components provide motion samples, the KNN model performs classification, and the feedback components update the OLED and buzzer. Wi-Fi and API components handle communication with FastAPI, while the repetition counter and dashboard data represent the user-visible results.

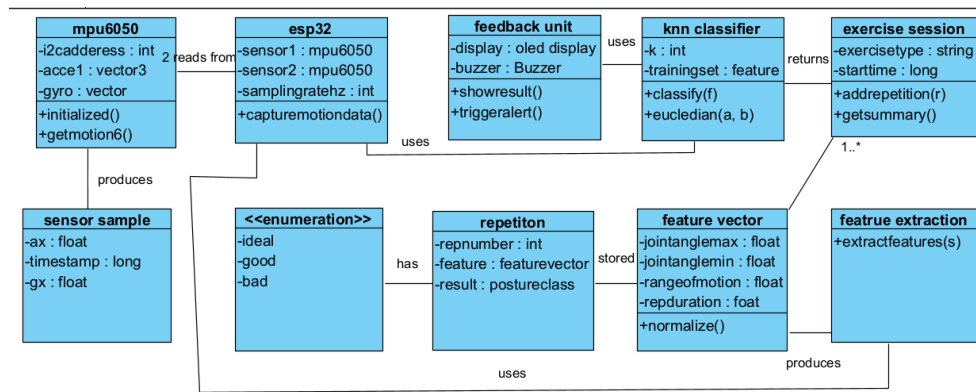


Figure 6.6: Class diagram of the Smart Gym Assistant software structure.

Although the firmware is implemented in embedded C++, the class diagram is a logical representation of its modules and responsibilities. It shows how the components cooperate without requiring the implementation to use a large software

framework.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter presents the experimental results of the Smart Gym Assistant. The KNN model was evaluated using a reproducible stratified validation split covering six classes: bicep bad, bicep good, bicep idle, squat bad, squat good, and squat idle. The experiments compared different values of k and different numbers of representative prototypes per label.

7.1 KNN Parameter Evaluation

Figure 7.1 compares validation accuracy for five values of k . Increasing the prototype count generally improves classification because the stored examples represent a wider range of motion patterns. The highest observed accuracy is approximately 89.06% with 128 prototypes per label and $k = 3$. The 64-prototype configuration used by the current embedded model achieves about 80.56% at $k = 5$, providing a smaller memory footprint for the ESP32.

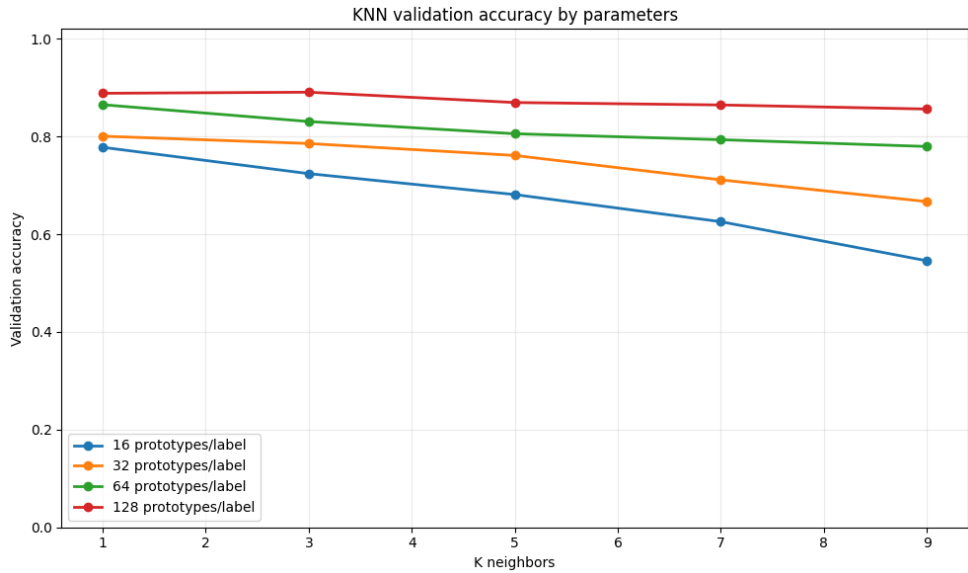


Figure 7.1: KNN validation accuracy for different parameter values.

The parameter-performance heatmap in Figure 7.2 gives the same comparison in tabular visual form. It shows that larger prototype sets are more accurate across the tested values of k , while excessively increasing k reduces accuracy for this

dataset. The final choice should therefore consider both validation performance and the memory and computation available on the ESP32.

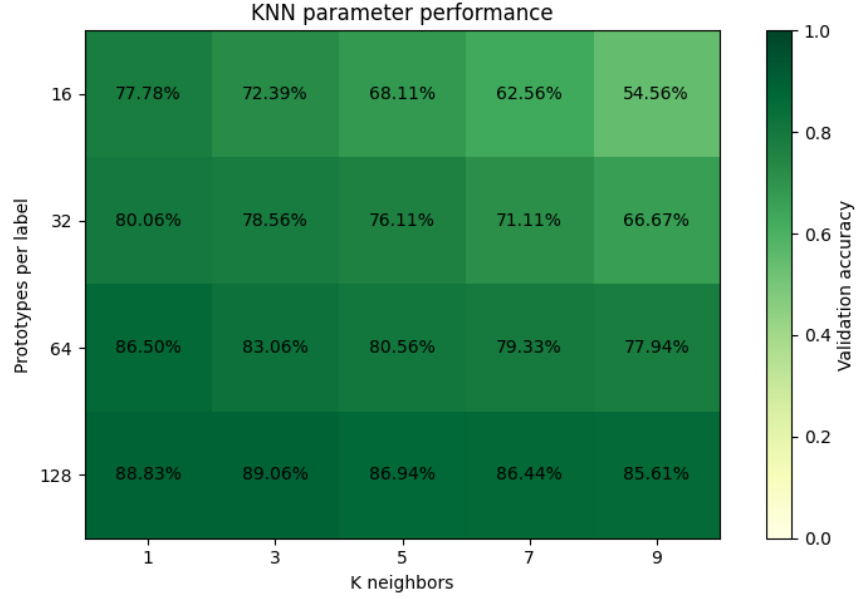


Figure 7.2: KNN parameter-performance heatmap.

7.2 Class-Wise Performance

The normalized confusion matrix in Figure 7.3 and the class-wise metrics give the following results:

- Bicep bad: 91.97% precision, 84.00% recall, and 87.80% F1-score.
- Bicep good: 84.66% precision, 88.33% recall, and 86.46% F1-score.
- Bicep idle: 91.51% precision, 97.00% recall, and 94.17% F1-score.
- Squat bad: 87.46% precision, 81.33% recall, and 84.28% F1-score.
- Squat good: 86.71% precision, 87.00% recall, and 86.86% F1-score.
- Squat idle: 92.06% precision, 96.67% recall, and 94.31% F1-score.

The results can be interpreted as follows:

- The idle classes are recognized most consistently, with F1-scores above 94%.
- Bicep idle has 97.0% recall and squat idle has 96.7% recall, showing that stationary samples are well separated from movement classes.

- The main confusion occurs between bad and good posture within the same exercise. Squat bad is classified as squat good in approximately 12.3% of samples, while squat good is classified as squat bad in approximately 10.0%.
- Squat bad has the lowest recall at 81.33% and the lowest F1-score at 84.28%, so incorrect squat posture is the most difficult class to detect.
- Bicep bad has high precision but lower recall, meaning that bad predictions are usually correct but some bad-posture samples are missed.
- Additional bad-posture samples, clearer posture boundaries, and consistent sensor placement could improve safety-related feedback.

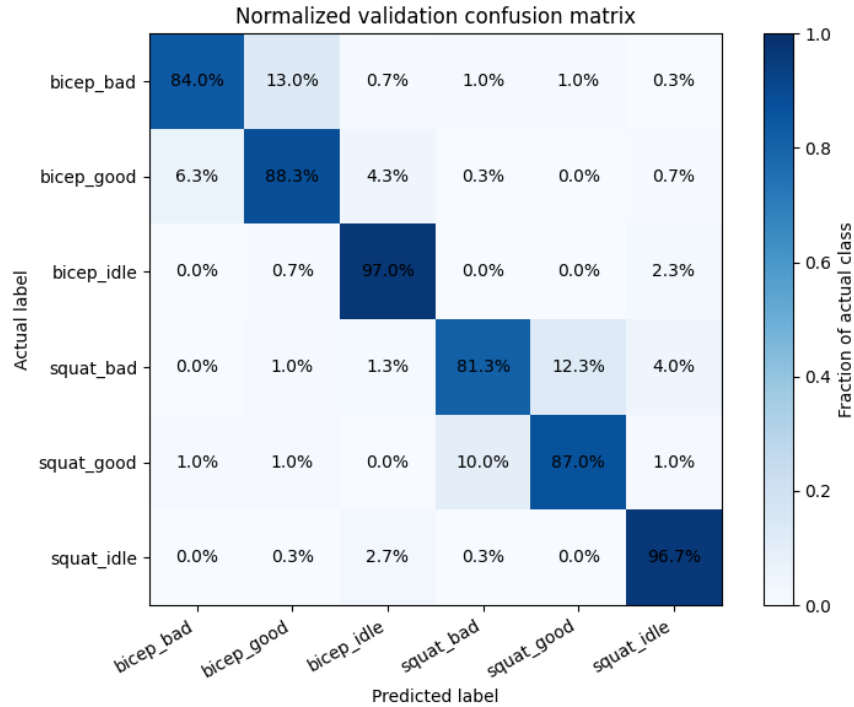


Figure 7.3: Normalized validation confusion matrix.

7.3 Discussion

The results demonstrate that a compact KNN classifier can distinguish exercise and posture states from twelve inertial features. Batch voting improves the stability of individual predictions, and local inference allows the ESP32 to provide OLED and buzzer feedback without waiting for the network.

The results also show a practical trade-off. More prototypes improve validation

accuracy but require additional program memory and distance calculations. The current 384-sample embedded model represents six labels with 64 samples each and uses $k = 5$, which is suitable for the available embedded resources. Additional training data from different users, improved posture labeling, and a later model export using the best validated parameters could improve generalization.

CHAPTER 8

CONCLUSION

This project presents a low-cost smart gym assistant for monitoring exercise movement and posture. Two MPU6050 sensors collect acceleration and gyroscope measurements, while the ESP32 prepares the sensor data and performs local K-nearest neighbors classification. The current implementation focuses on bicep curls and squats and can identify the exercise and posture class from labeled motion data.

The system combines live batch prediction with repetition-counting logic and immediate feedback through an OLED display and buzzer. In collection mode, labeled sensor batches are sent to the FastAPI service for storage and later model training. In live mode, the ESP32 keeps the prediction process local and publishes the latest exercise, posture, confidence, and repetition information to the web monitor.

The prototype demonstrates that wearable inertial sensors and a compact machine-learning model can provide practical exercise feedback on resource-constrained hardware. Its performance still depends on consistent sensor placement, representative training data, and balanced posture classes. Future work can include additional exercises such as push-ups, more users and movement styles, improved repetition-counting thresholds, and comparison with quantized neural-network models.

CHAPTER 9

LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Limitations

The current Smart Gym Assistant demonstrates exercise and posture recognition using wearable inertial sensors, but several limitations remain:

- The implemented classifier supports only bicep curls and squats with the available labeled classes.
- Recognition depends on consistent sensor placement and orientation on the user's body.
- The training data may not represent every user's movement style, body type, speed, or range of motion.
- Bad and good posture classes can overlap, particularly for squats, which reduces recall for incorrect posture detection.
- The ESP32 uses a compact prototype set to limit memory and computation, so it may not capture every variation in the motion data.
- Live dashboard monitoring depends on Wi-Fi availability and communication with the FastAPI server.

9.2 Future Improvements

The following improvements can extend the accuracy, usability, and scalability of future versions of the system.

9.2.1 Support for Additional Exercises

Additional exercises such as push-ups, lunges, shoulder presses, and jumping jacks can be added by collecting representative labeled data for each exercise and posture condition. The training pipeline and embedded label definitions can then be extended without changing the basic sensing architecture.

9.2.2 Training with More Users and Motion Styles

Future datasets should include multiple users, sensor placements, movement speeds, and body-motion variations. This would improve generalization and reduce the risk that the classifier performs well only for the user who collected the original data.

9.2.3 Improved Posture Detection

More carefully defined posture boundaries and additional bad-posture examples could improve recall for safety-related classes such as `squat_bad`. Future work could also combine temporal features, movement ranges, and exercise-specific rules with the KNN prediction.

9.2.4 Optimized Embedded Model

The model can be regenerated using the best validated prototype count and value of k , provided the resulting memory and processing requirements remain suitable for the ESP32. Quantization, dimensionality reduction, or a compact alternative model could further reduce inference cost.

9.2.5 Offline Data Buffering and Reliable Communication

The ESP32 could temporarily buffer records when Wi-Fi is unavailable and upload those records after reconnection. Retry handling, request status reporting, and server-side validation could make collection and live monitoring more reliable.

9.2.6 User Interface Enhancements

The dashboard could provide session history, exercise summaries, repetition trends, class-wise confidence, and calibration guidance. The OLED interface could also show clearer status messages for sensor placement, Wi-Fi state, and posture feedback.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2021.
- [2] A. Geron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.
- [3] W. McKinney, *Python for Data Analysis*, 3rd ed. O'Reilly Media, 2022.
- [4] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [5] F. Wang et al., “Wearable sensor-based human activity recognition using machine learning: A survey,” *IEEE Access*, vol. 7, pp. 170 529–170 547, 2019.
- [6] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, “Activity recognition using cell phone accelerometers,” *ACM SIGKDD Explorations*, vol. 12, no. 2, pp. 74–82, 2011.
- [7] O. D. Lara and M. A. Labrador, “A survey on human activity recognition using wearable sensors,” *IEEE Communications Surveys & Tutorials*, vol. 15, no. 3, pp. 1192–1209, 2013.
- [8] A. Bulling, U. Blanke, and B. Schiele, “A tutorial on human activity recognition using body-worn inertial sensors,” *ACM Computing Surveys*, vol. 46, no. 3, 2014.
- [9] F. J. Ordonez and D. Roggen, “Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition,” *Sensors*, vol. 16, no. 1, 2016.
- [10] M. Banzi and M. Shiloh, *Getting Started with Arduino*, 4th ed. Maker Media, 2022.
- [11] *ESP32 Series Datasheet*, Espressif Systems, 2024.
- [12] *MPU-6000 and MPU-6050 Product Specification*, InvenSense, 2013.

- [13] L. Bao and S. S. Intille, “Activity recognition from user-annotated acceleration data,” in *Pervasive Computing*. Springer, 2004, pp. 1–17.
- [14] N. Ravi, N. Dandekar, P. Mysore, and M. L. Littman, “Activity recognition from accelerometer data,” in *Proceedings of the Seventeenth Conference on Innovative Applications of Artificial Intelligence*, 2005, pp. 1541–1546.
- [15] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Prentice Hall, 2010.
- [16] S. Ramirez, “Fastapi documentation,” <https://fastapi.tiangolo.com/>, 2024.